

Binary Content Protocol (BCP)

Version 1 | Default TCP port 9000 | Network byte order

1. Purpose and connection model

BCP transfers named files over a reliable TCP byte stream. A client opens one connection and may exchange any number of request/response pairs on it. Each request uses a non-zero stream identifier and its response repeats that identifier. Version 1 clients send a request and read its response before sending the next, so responses stay in request order.

TCP has no message boundaries. A receiver reads exactly 12 header bytes, obtains the payload length, then reads exactly that many payload bytes. A short `recv()` is normal. End-of-file in either part is a protocol error.

2. Fixed frame header

Offset	Width	Field	Meaning
0	2	magic	ASCII BC (42 43 hex)
2	1	version	1 for this specification
3	1	type	1 request; 2 response; other values unrecognized
4	1	flags	bit 0 END_STREAM; bits 1-7 zero
5	1	header count	fields at the start of the payload
6	2	stream id	unsigned correlation id; 0 reserved for connection errors
8	4	payload length	bytes after this header; implementation maximum 8 MiB

Two magic bytes reject the wrong protocol early. Eight-bit version and type fields leave extension space. A 16-bit stream id is sufficient for a sequential client and allows future multiplexing. The 32-bit length makes every frame skippable; the 8 MiB implementation limit bounds memory use. END_STREAM is set on every version 1 frame because each message fits in one frame.

Forward compatibility: a receiver that meets an unrecognized frame type MUST read and discard exactly its declared payload length, then continue at the next frame. Header count and payload contents are opaque for that frame.

3. Header block and body

The payload begins with exactly header-count fields. Any bytes left after those fields are the body.

```
indexed: code (1) | value length (2) | UTF-8 value
literal: 0 (1) | name length (1) | ASCII name | value length (2) | UTF-8 value
```

Lengths are unsigned big-endian integers. Literal names contain 1-255 lowercase ASCII bytes. Values contain at most 65,535 bytes. Duplicate names are invalid. Code 0 permits uncommon names without shared dynamic state.

Binary Content Protocol (BCP)

Version 1 - Messages and receiver behavior

4. Static header-name table

Code	Name	Direction	Use
1	:method	client	request method; GET in version 1
2	:path	client	origin-form resource path
3	host	client	server authority
4	user-agent	client	client identifier
5	accept	client	acceptable media types
6	:status	server	three-digit decimal result
7	content-length	server	body size in decimal bytes
8	content-type	server	body media type
9	server	server	server identifier
10	connection	server	keep-alive or close

5. Request and response

A request has type 1 and an empty body. It requires :method=GET, an origin-form :path beginning with /, and host. user-agent and accept are normally included. A response has type 2 and repeats the stream id. It requires decimal :status, decimal content-length, content-type, server, and connection. Content length must equal the body byte count.

The server percent-decodes the path and maps it beneath its configured root. It rejects NUL, backslash, malformed escape, and .. traversal. A directory maps to index.html. Query text is not part of the filesystem path.

6. Status and error handling

Status	Meaning
200	The response body is the requested file.
400	The frame, required headers, or path is malformed.
404	The mapped file is unavailable.
405	The request method is not supported.

The client exits non-zero for 4xx or 5xx. Bad magic, unsupported version, unknown flag bits, excessive length, truncation, and invalid header encoding are connection-level errors. When framing is still safe, the server sends status 400 on stream 0 with connection=close and closes. Otherwise it closes immediately. Unknown frame types are not errors and follow the mandatory skip rule.

7. Connection lifetime

One TCP connection carries all requests for the same authority. Either peer may close an idle connection; the reference server waits 15 seconds. Normal closure occurs only between complete frames. The version 1 client does not pipeline, but the stream id leaves room for a later version to define multiplexing without changing the fixed header.

Receiver checklist

1. Read 12 bytes exactly. 2. Validate magic, version, flags, and length. 3. Read payload exactly. 4. Skip unknown types without parsing. 5. Decode exactly header-count fields. 6. Treat the remainder as body. 7. Validate required fields and body length. 8. Continue at the next 12-byte header.